

Artificial Neural Network (ANN) Model Architecture

Customer Churn Prediction – Stage 3

1. Purpose

The purpose of the Artificial Neural Network (ANN) model is to predict whether a customer is likely to churn. The model is designed as a **binary classification model**, where:

- **0 = No churn**
- **1 = Churn**

The ANN uses the prepared customer features from the data-preprocessing stage to learn patterns associated with customer churn and generate a probability of churn for each customer.

2. ANN Architecture

The selected ANN architecture consists of an input layer, two fully connected hidden layers with dropout regularisation, and a single-neuron output layer.

Architecture Overview

Input Layer → Dense Layer (32) → Dropout (20%) → Dense Layer (16) → Dropout (10%) → Output Layer (1)

Layer	Type	Units	Activation	Purpose
1	Input	Number of prepared features	—	Receives the processed customer data
2	Hidden	32	ReLU	Learns non-linear relationships between customer features
3	Regularisation	—	—	20% dropout to reduce overfitting
4	Hidden	16	ReLU	Learns higher-level patterns from the previous layer
5	Regularisation	—	—	10% dropout to provide additional regularisation

Layer	Type	Units	Activation	Purpose
6	Output	1	Sigmoid	Produces the predicted probability of customer churn

3. ANN Model Code

The ANN architecture is implemented using the following code:

```
model = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(32, activation="relu"),
    Dropout(0.20),
    Dense(16, activation="relu"),
    Dropout(0.10),
    Dense(1, activation="sigmoid"),
])
```

Explanation

The Input layer automatically accepts the number of features available in the prepared training dataset.

The first Dense layer contains **32 neurons** and uses the **ReLU activation function**. This layer allows the model to learn non-linear relationships between the customer characteristics and churn.

A **20% dropout layer** follows the first hidden layer. Dropout randomly excludes a proportion of neurons during training and helps reduce the risk of overfitting.

The second hidden layer contains **16 neurons**, again using ReLU activation. The smaller second layer allows the network to progressively learn a more compact representation of the customer data.

A second dropout layer with a rate of **10%** provides additional regularisation.

The final layer contains **one neuron with sigmoid activation**. Since churn is a binary classification problem, the sigmoid function produces a value between 0 and 1 that can be interpreted as the predicted probability of churn.

4. Input Layer

The input layer receives the customer features after data preprocessing.

The categorical variables are converted into numerical representations using **one-hot encoding**, while numerical variables are standardised.

The number of input neurons is therefore determined by:

```
X_train.shape[1]
```

This approach ensures that the input layer automatically matches the number of features generated during preprocessing.

5. Hidden Layers

First Hidden Layer

```
Dense(32, activation="relu")
```

The first hidden layer contains **32 neurons** and uses the ReLU activation function.

Its purpose is to identify relationships and patterns within the customer features that may be associated with churn.

First Dropout Layer

```
Dropout(0.20)
```

A dropout rate of **20%** is applied after the first hidden layer.

This provides regularisation and helps reduce the risk of the ANN becoming too closely fitted to the training data.

Second Hidden Layer

```
Dense(16, activation="relu")
```

The second hidden layer contains **16 neurons** and also uses ReLU.

The reduction from 32 to 16 neurons provides a progressively smaller representation of the learned information before the final prediction.

Second Dropout Layer

```
Dropout(0.10)
```

A **10% dropout rate** provides additional regularisation before the output layer.

6. Output Layer

The output layer is:

```
Dense(1, activation="sigmoid")
```

The output layer contains a single neuron because the model predicts one binary outcome: whether a customer will churn.

The sigmoid activation function produces a probability between **0 and 1**.

For the current model, a threshold of **0.5** is used:

- Probability $\geq 0.5 \rightarrow$ predicted churn
- Probability $< 0.5 \rightarrow$ predicted non-churn

The probability itself can also be used to rank customers according to their predicted churn risk.

7. Model Training Configuration

The ANN is compiled using:

```
model.compile(  
    optimizer="adam",  
    loss="binary_crossentropy",  
    metrics=["accuracy"]  
)
```

The training configuration is:

Setting	Configuration
Optimiser	Adam
Loss function	Binary cross-entropy
Training metric	Accuracy
Maximum epochs	100
Batch size	32
Validation split	20%

Setting	Configuration
Early stopping	Patience of 8
Best weights	Restored after early stopping

Optimiser

The **Adam optimiser** is used to update the model's weights during training.

Loss Function

Binary cross-entropy is used because the model is performing binary classification.

Early Stopping

The model uses:

```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=8,
    restore_best_weights=True
)
```

The validation loss is monitored during training. If it does not improve for eight consecutive epochs, training is stopped and the best-performing weights are restored.

This helps reduce unnecessary training and the risk of overfitting.

8. Data Leakage Prevention

A key consideration in developing the predictive model is preventing **data leakage**.

The dataset is divided into training and testing data before the preprocessing parameters are learned.

The preprocessing process is fitted using the training data only. The test data is then transformed using the preprocessing parameters learned from the training set.

This prevents information from the test dataset from influencing the model during training.

The dataset uses an **80% training and 20% testing split**, with stratification to maintain the distribution of churn and non-churn customers.

This provides a more reliable evaluation of how the ANN performs on unseen customer data.

9. Predictive Performance Optimisation

The selected architecture is designed to balance model complexity and predictive performance.

The **32-neuron first hidden layer** provides enough capacity to learn relationships within the prepared customer data, while the **16-neuron second layer** progressively reduces the representation before prediction.

The two dropout layers provide regularisation to reduce overfitting.

The sigmoid output is appropriate for binary churn prediction, while binary cross-entropy provides an appropriate loss function for the classification task.

Early stopping further supports generalisation by stopping training when validation performance no longer improves.

However, this architecture should be described as the **selected architecture for the project**, rather than claiming it is mathematically proven to be the globally optimal ANN architecture. Demonstrating a globally optimal architecture would require systematic hyperparameter tuning and comparison of alternative architectures using a consistent validation procedure.

10. Model Evaluation

The ANN is evaluated using several performance measures:

Accuracy

Measures the overall percentage of correctly classified customers.

Precision

Measures how many customers predicted to churn actually churned.

Recall

Measures how many of the customers who actually churned were successfully identified.

Recall is particularly important for customer churn because failing to identify a customer who is likely to leave could result in a missed retention opportunity.

F1-Score

Provides a balance between precision and recall.

ROC-AUC

Measures how well the model distinguishes between churn and non-churn customers across different probability thresholds.

A **confusion matrix** is also used to examine true positives, true negatives, false positives and false negatives.

11. Expected Model Output

After training, the ANN produces a churn probability for each customer in the test dataset.

The prediction code is:

```
probabilities = model.predict(
    X_test.to_numpy(),
    verbose=0
).ravel()

predictions = (
    probabilities >= 0.5
).astype(int)
```

The probability provides a useful risk score that can be used to prioritise customers for potential retention activities.

12. Architecture Summary

The final ANN architecture is:

```
Input
↓
Dense Layer – 32 neurons – ReLU
↓
Dropout – 20%
↓
```

Dense Layer – 16 neurons – ReLU

↓

Dropout – 10%

↓

Output Layer – 1 neuron – Sigmoid

↓

Churn Probability

The model combines **non-linear learning through ReLU hidden layers, dropout regularisation, sigmoid-based binary prediction, Adam optimisation, binary cross-entropy loss, and early stopping.**

Together, these components provide a structured ANN model for predicting customer churn while controlling the risk of overfitting and data leakage.